CUPS Printer Discovery Exploitation and Attack Detection

An Isolated Scapy-Based Security Lab

Jananka — November 2026

https://github.com/jananka259/cups-security-lab

Abstract

This report examines how a vulnerable CUPS service handles printer discovery messages. We built an isolated Docker network with a Scapy attacker, a vulnerable `cups-browsed` victim, and a virtual IPP printer. We generated normal and modified advertisements, recorded traffic, tested a Scapy detection tool, and applied an allowlist defence. Normal discovery led to a TCP connection and IPP requests, although printer queue creation failed. An unauthorized advertisement was sent and detected, but the available captures do not establish that the victim contacted the attacker-advertised endpoint. The distinction between sending an advertisement, processing it, connecting to a printer, and creating a queue is central to the results.

1 Introduction and background

CUPS manages printing on Unix-like systems. Its legacy `cups-browsed` discovery mechanism can receive printer announcements over UDP port 631. A discovery message contains an IPP printer URI and descriptive fields. If a service trusts an unverified announcement, it may contact a printer address chosen by another machine. The 2024 CUPS vulnerability family brought attention to this trust boundary [2].

The assignment asks for a controlled demonstration of printer discovery, a Scapy attack tool, a detector, several packet variations, evidence-based evaluation, and a mitigation [1]. Remote code execution is not required and was not attempted.

2 Lab environment

Table 1: Isolated Docker lab network (`172.28.77.0/24`).

Role	IP address	Software and function
Attacker	`172.28.77.10`	Ubuntu-based container, Python, Scapy, packet capture
Victim	`172.28.77.20`	Vulhub `cups-browsed:2.0.1` container
Virtual printer	`172.28.77.30`	OpenPrinting `ippserver`, TCP port 631

The Docker Compose network sets `internal: true`, uses static addresses, and publishes no service ports to the host. The attacker container receives `NET_RAW` for controlled packet construction. The victim image uses `linux/amd64` on the macOS Docker Desktop host. No public target or physical printer was used. The address `172.28.77.40` was used as an *advertised attacker-controlled endpoint*; it was not the configured virtual printer at `.30`.

Key configuration is in `docker-compose.yml`, `Dockerfile.printer`, and the mitigation override. Recorded configuration and container status are in `evidence/logs/` in the repository.

3 Attack tool: `cups_attack.py`

The attack tool creates legacy CUPS browse advertisements using Scapy's `IP / UDP / Raw` layers. A typical payload has five fields: a message type, printer state, IPP URI, display name, and information field. Source and destination UDP ports are 631.

```
0 3 ipp://172.28.77.40:631/printers/lab "Lab Printer" "Lab Printer"
```

The tool takes command-line options for victim, advertiser, printer IP and port, queue/name, message variation, repetition, interval, PCAP path, and log path. `--dry-run` writes packets without sending them; live transmission requires `--authorize-lab`. Input checks keep the destination fixed to the isolated victim and restrict addresses to the lab subnet. Four packet modes are implemented: baseline, missing URI, invalid port, and malformed fields. Repetition is also exercised in the experiments.

4 Detection tool: `cups_detector.py`

The detector reads an offline PCAP with Scapy's `rdpcap` or monitors live packets with `sniff`. It parses browse fields, checks separate allowlists for sender and advertised printer, records repeated messages, and correlates TCP traffic with advertised endpoints. Its output is structured JSON or CSV.

Table 2: Implemented detection rules.

Rule	Meaning
`DISCOVERY-001`	Advertisement sent by an unauthorized source
`DISCOVERY-002`	Printer host or port not on the permitted list
`DISCOVERY-003`	Invalid URI, port, field count, or message format
`DISCOVERY-004`	Same advertisement repeated at least three times within 60 seconds
`IPP-001`	TCP traffic correlated with an advertised unauthorized IPP endpoint

The TCP correlation rule exists in the implementation, but these experiments did *not* produce a real `IPP-001` finding for endpoint `.40`. This is a coverage limitation of the evidence, not a failure to define the rule.

5 Experiments and observations

5.1 Experiment 1: legitimate discovery

The authorized virtual printer was at `172.28.77.30:631`. Its advertisement identified the printer URI below:

```
ipp://172.28.77.30:631/printers/lab
```

The capture `pcap_files/real/legitimate-clean.pcap` contains discovery traffic, a TCP three-way handshake, and subsequent IPP messages. This establishes that the victim initiated communication with the legitimate advertised endpoint.

After the handshake the victim requested `Get-Printer-Attributes`. The response reported an IPP `client-error-not-found` for the requested `/printers/lab` resource, even though the HTTP transport returned `200 OK`. The simulated printer's actual endpoint was `/ipp/print`; consequently the evidence supports *discovery and IPP communication*, not successful queue creation or printing.

5.2 Experiment 2: unauthorized discovery

The attacker at `172.28.77.10` sent a discovery advertisement to victim `172.28.77.20`, naming `172.28.77.40:631` as the printer. The real `unauthorized.pcap` records that UDP datagram; the detector generated two alerts for unauthorized sender and suspicious advertised endpoint.

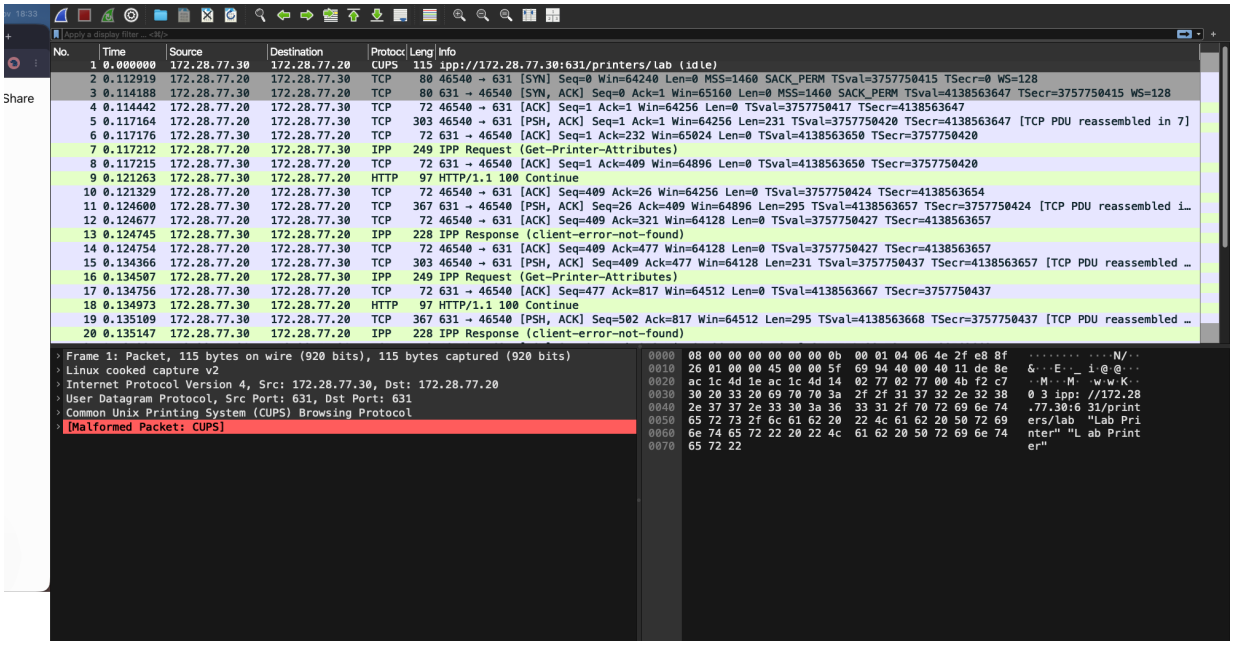


Figure 1: Packet list from the legitimate capture, including discovery and TCP/IPP traffic.

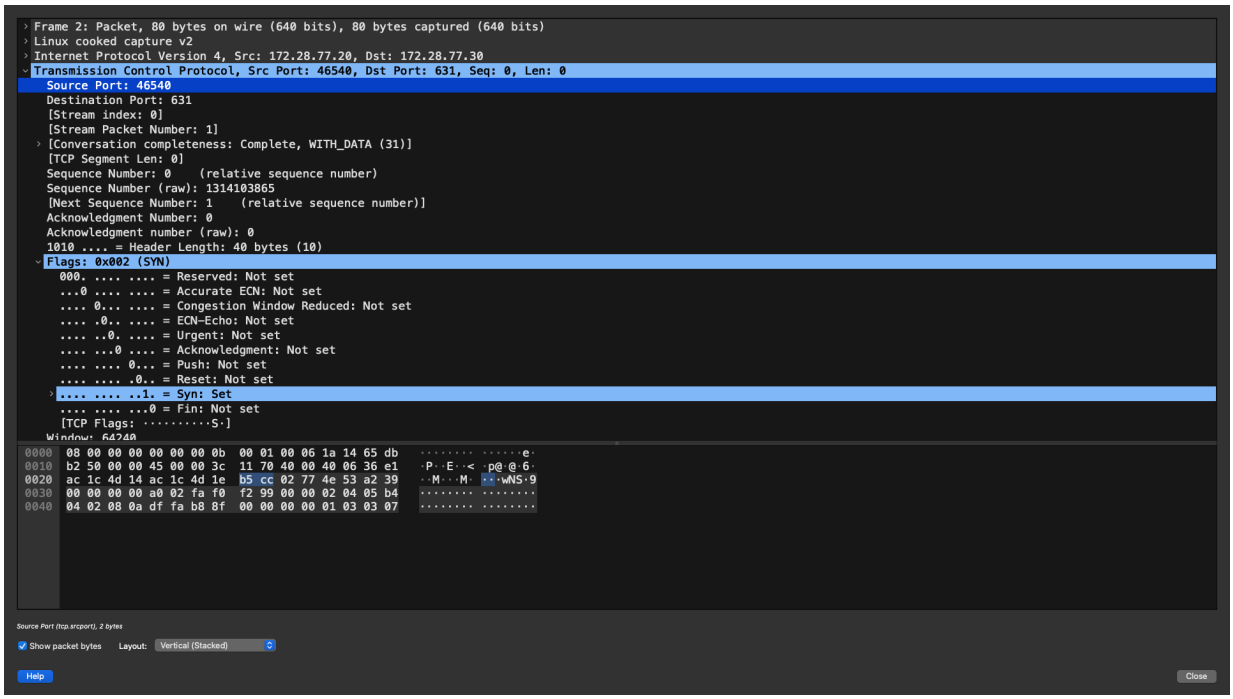


Figure 2: TCP SYN from the victim `.20` to the legitimate printer `.30:631`.

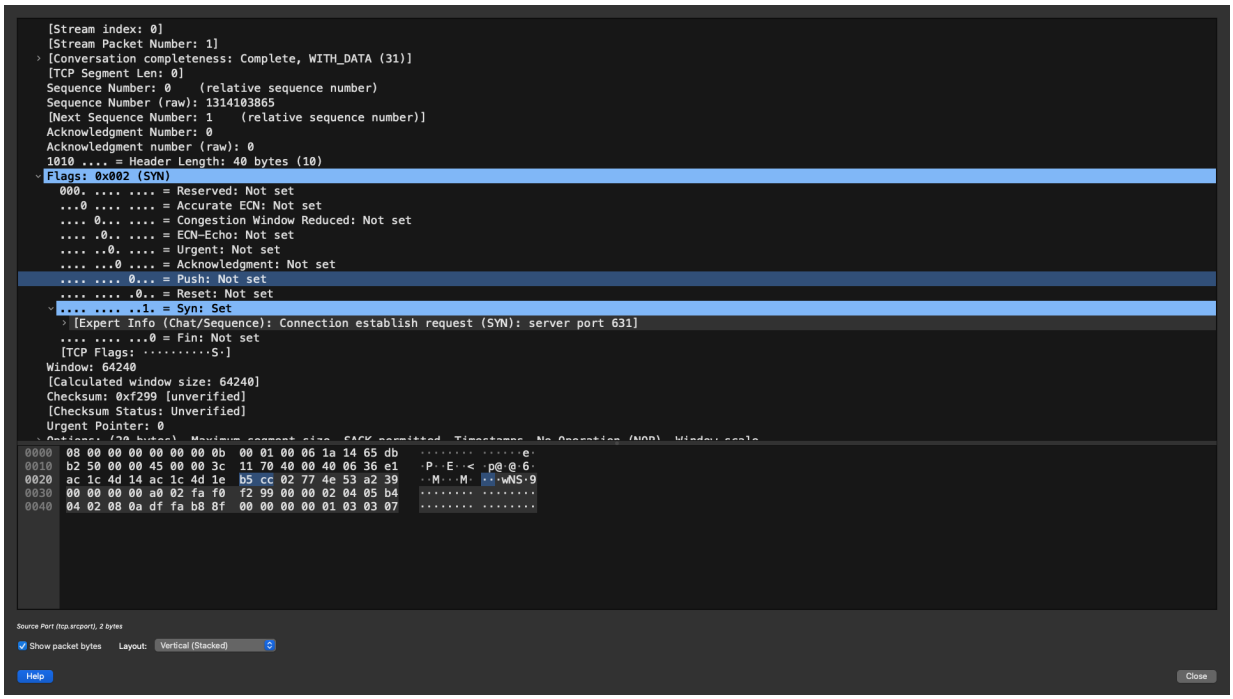


Figure 3: TCP SYN/ACK returned by the legitimate printer, confirming a response to the connection request.

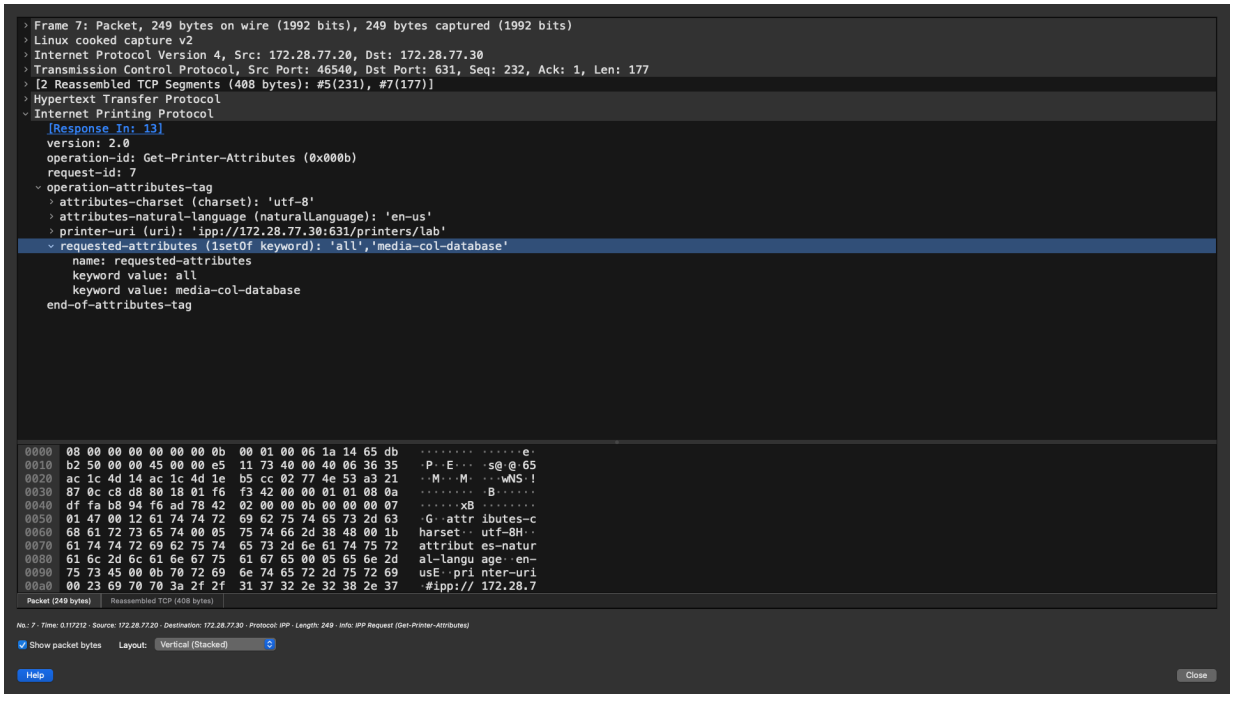


Figure 4: IPP `Get-Printer-Attributes` request referring to the legitimate printer URI.

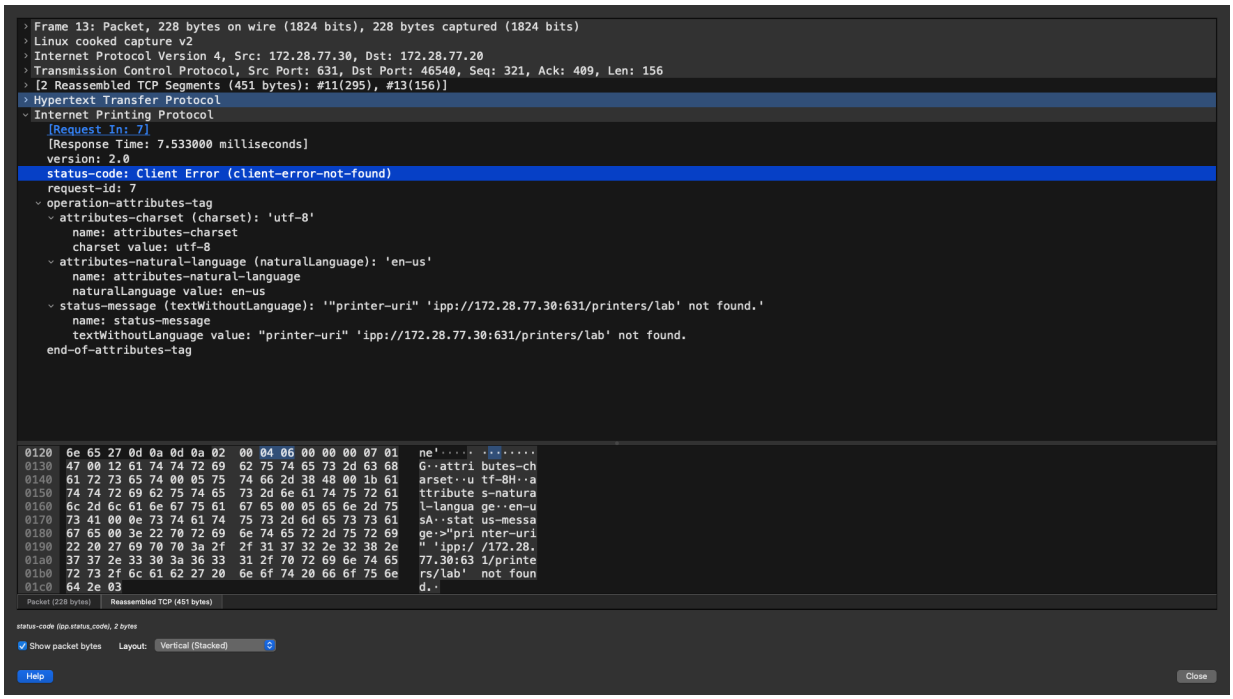


Figure 5: IPP response reports that `/printers/lab` was not found.

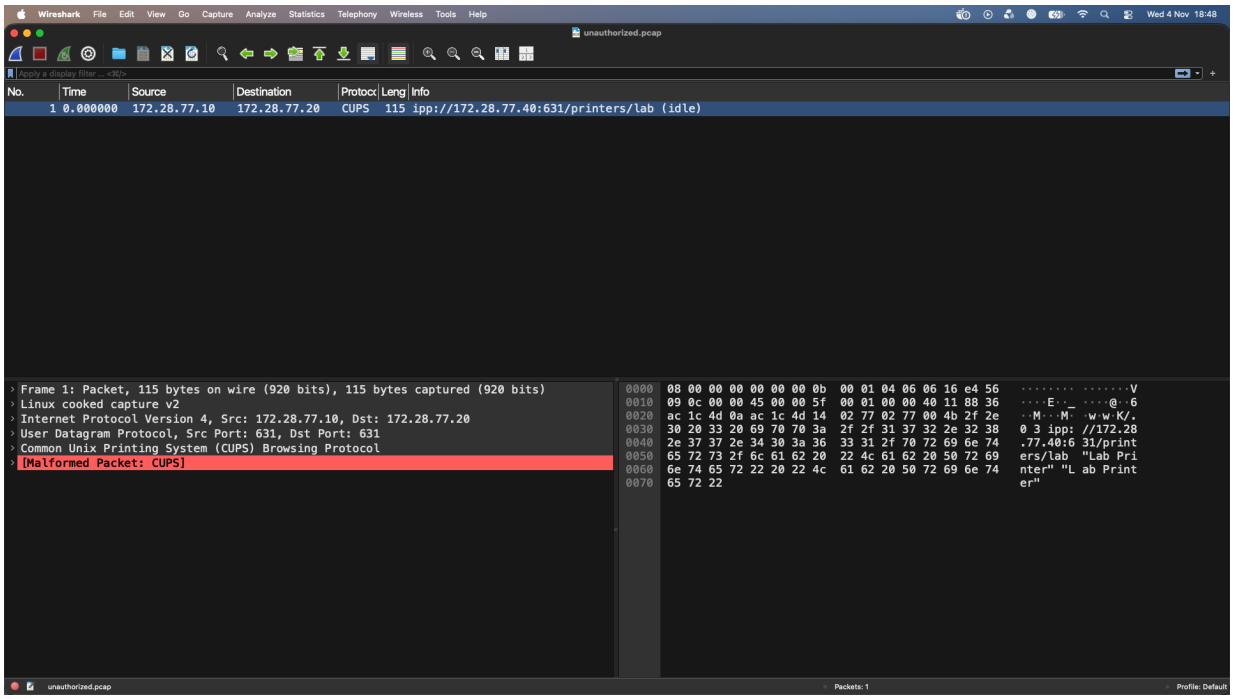


Figure 6: Unauthorized CUPS browse message from `.10` to `.20`, advertising printer `.40`. Wireshark also marks the legacy payload as malformed in its CUPS dissector.

Important outcome: the seven existing real PCAPs were checked for packets with source `172.28.77.20` and destination `172.28.77.40`. None were found. Thus these captures *do not demonstrate* a victim connection to the attacker-advertised endpoint. They also do not prove that no attempt could occur outside their capture scope or time window. The normal `.30` connection cannot be treated as proof of a `.40` connection.

5.3 Experiment 3: manipulated advertisements

Table 3: Measured detector output from real capture files.

Capture / variation	Packets	Alerts	Main result
`legitimate-clean.pcap`	30	0	Authorized baseline
`unauthorized.pcap`	1	2	`001`, `002`
`repeated.pcap`	3	7	Repetition plus sender/endpoint alerts
`invalid-port.pcap`	1	1	`003`: port `99999`
`malformed.pcap`	1	1	`003`: incorrect field count

The three manipulated cases were repetition, invalid port, and malformed fields. The code also supports a missing-URI variant, although it is not counted as a separately observed real-capture experiment here.

5.4 Experiment 4: detection evaluation

The legitimate capture produced zero alerts, so no false positive was observed in that baseline. Each of the four attack-oriented capture files produced the expected rule class, so no missed case was observed among this small test set. These results cannot establish a general false-positive or false-negative rate: the captures were short and intentionally constructed. The correlation rule `IPP-001` was not validated against a captured unauthorized connection.

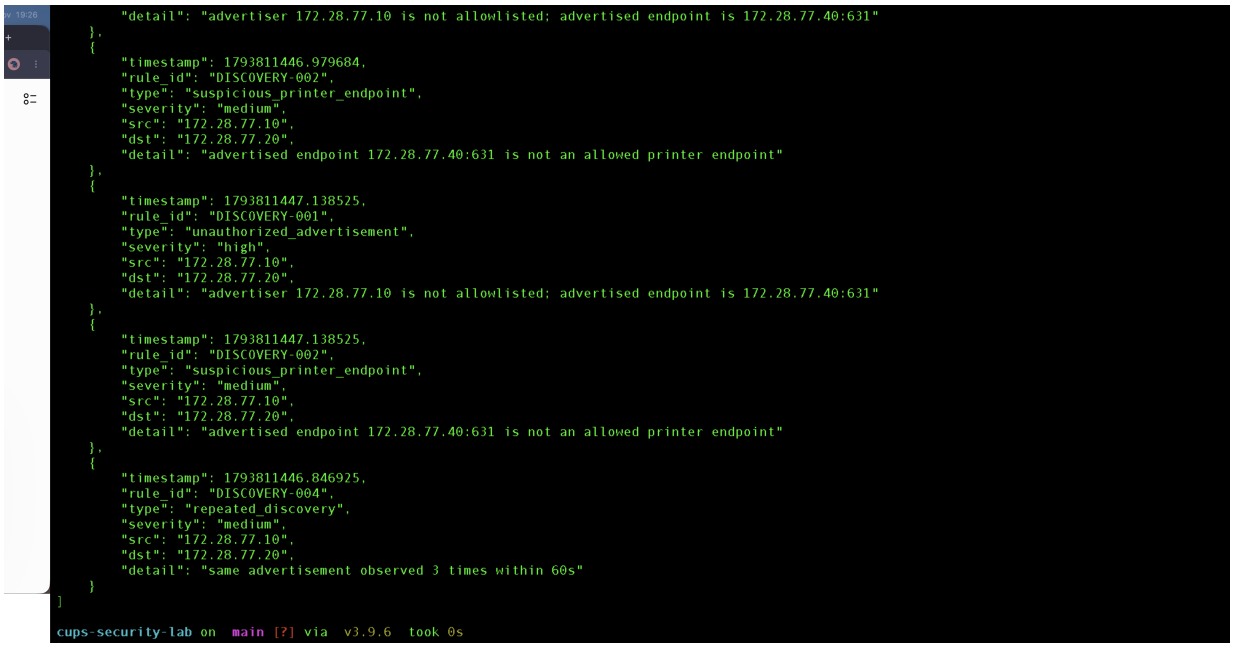


Figure 7: Detector JSON output showing unauthorized endpoint alerts and a repeated-discovery alert.

6 Mitigation and before/after comparison

The mitigation in `mitigation/cups-browsed.conf` restricts accepted browse advertisements to the legitimate source:

```
BrowseRemoteProtocols cups
BrowseOrder Allow,Deny
BrowseAllow 172.28.77.30
```

Before mitigation, the unauthorized advertisement was transmitted and observed in a capture. After applying the allowlist, the victim log explicitly recorded a rejection: `browse packet from 172.28.77.10 disallowed`. It also recorded an accepted browse packet from `172.28.77.30` and subsequent IPP attribute requests. The detector still flags the transmitted attacker packet: an alert indicates suspicious network input, whereas the allowlist controls how the victim handles it. This is why detection and prevention should be evaluated separately.

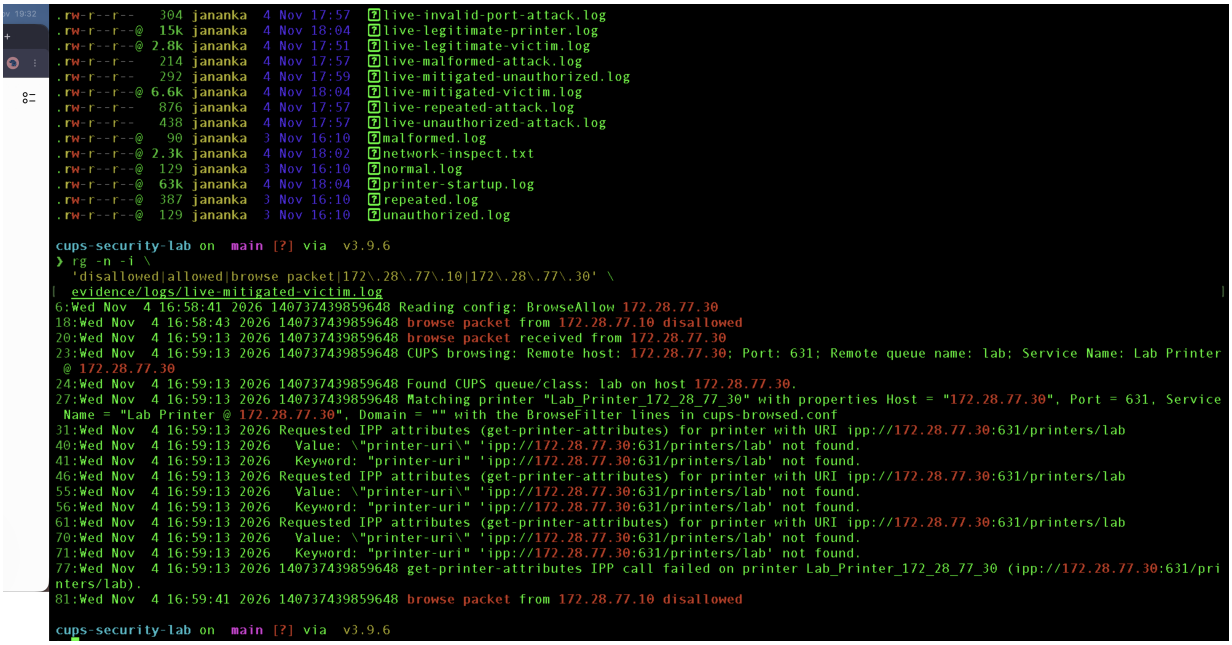


Figure 8: Mitigated victim log: unauthorized `.10` rejected, legitimate `.30` processed.

Other possible defences include limiting UDP/631 to trusted networks, disabling unneeded browse functionality, patching vulnerable packages, and network segmentation. Only the allowlist is claimed as implemented and observed in this lab.

7 Verification, limitations, and ethics

The project reports 11 passing Python tests. The tools were also tested against real captures, with JSON detection outputs under `evidence/detection_results/`. Packet evidence and victim/printer logs are kept in `pcap_files/real/` and `evidence/logs/`. The five detector command outputs used in Table 3 were recorded by the investigator; this report package does not independently rerun the Docker experiments.

Several limitations matter to interpretation. First, the real attacker advertisement capture contains only the sent datagram; there is no complete victim-to-`.40` connection trace. Second, the legitimate virtual IPP printer had a URI mismatch that prevented queue creation. Third, Wireshark may mark legacy CUPS browse messages as malformed even when a URI can be extracted. Finally, these few controlled cases do not test every traffic format or network condition.

All testing was confined to private Docker containers, with no external targets, physical printers, payloads for code execution, or persistence. The live packet tool requires an explicit lab authorization flag. The study is intended to understand a trust boundary and evaluate practical detection and defensive configuration.

8 Conclusion

The lab demonstrates the complete normal discovery-to-IPP communication path for a legitimate virtual printer. It also demonstrates creation and detection of unauthorized, repeated, and malformed discovery messages. The allowlist rejects the unauthorized source while allowing the legitimate sender. The available evidence does not show the victim contacting the attacker-advertised `.40` endpoint, and queue creation for the legitimate `.30` endpoint was unsuccessful. Reporting these outcomes separately gives a reproducible and appropriately limited answer to the assignment.

References

[1] Course assignment, *CUPS Printer Discovery Exploitation and Attack Detection Using Scapy*, `assignment.md` in the submitted repository, 2026.

[2] Vulhub, *cups-browsed vulnerability lab*, https://github.com/vulhub/vulhub/tree/master/cups-browsed/CVE-2024-47177.

[3] Scapy documentation, https://scapy.readthedocs.io/.

[4] Debian manual page, *cups-browsed.conf(5)*, https://manpages.debian.org/bookworm/cups-browsed/cups-browsed.conf.5.en.html.